

Take-home: pricing service

Hi, thanks for taking the time on this.

We're Asanbar, a freight marketplace. Before a shipment gets paid, we quote it, take our commission, and freeze the numbers in a snapshot. Your job is to build a small version of that **pricing service**: standalone repo, no access to our code needed.

Plan on **4-6 hours**. Send it back within **3 days** if you can. If you go over, note how long it actually took; we're not timing you with a stopwatch.

What you're building

Rough flow:

1. Someone posts shipment inputs → service returns a quote (gross, commission, driver net).
2. Later they confirm → latest quote becomes an immutable snapshot.
3. Confirm publishes `price_snapshot.created` on a Redis stream so payment can pick it up.

Money is always **IRR as integers**. No floats, no Decimal for amounts. This should always hold:

```
driver_net_amount + commission_amount == gross_amount
```

Stack: **Python 3.12**, **FastAPI**, **PostgreSQL** (schema `pricing`), **Alembic**, **Redis Streams**, **pytest**. API lives under `/api/v1/pricing`. Add `GET /health` and `GET /ready` (ready checks DB).

Skip auth. No JWT for this exercise.

Pricing logic

Hardcode the rate card, put it in YAML, seed it in Postgres. Up to you:

```
general      + trailer    → 12_000 IRR/km, 500_000 per extra stop
general      + single     → 10_000 IRR/km, 400_000 per extra stop
refrigerated + trailer    → 15_000 IRR/km, 600_000 per extra stop
```

Unknown cargo/vehicle combo → **422**, error code `UNKNOWN_RATE_CARD`.

Calculation:

```
base = distance_km * rate_per_km + stop_fee * max(0, stop_count - 1)
gross = base
commission = round(gross * commission_rate_bps / 10_000) # integer math
driver_net = gross - commission
```

Seed one commission rule: name `"default"`, `commission_rate_bps = 1000` (10%), active. No admin API needed.

Validate inputs: `distance_km > 0`, `stop_count >= 1`.

Endpoints

Errors look like this everywhere:

```
{
  "error": {
    "code": "QUOTE_NOT_FOUND",
    "message": "...",
    "correlation_id": "..."
  }
}
```

If the client sends `X-Correlation-Id`, pass it through on errors.

POST `/api/v1/pricing/shipments/{shipment_id}/quote`

Body:

```
{
  "inputs": {
    "distance_km": 450,
    "stop_count": 2,
    "cargo_type": "general",
    "vehicle_type": "trailer"
  },
  "force_recalculate": false
}
```

- `force_recalculate: false` : same shipment + same inputs → return existing quote (same `quote_id`).
- `force_recalculate: true` : bump `quote_version` (1, 2, 3...).

Response shape (amounts are examples; compute the real ones):

```
{
  "quote_id": "...",
  "shipment_id": "...",
  "quote_version": 1,
  "inputs": { "distance_km": 450, "stop_count": 2, "cargo_type": "general", "vehicle_type": "trailer" },
  "gross_amount": 5900000,
  "commission_amount": 590000,
  "driver_net_amount": 5310000,
  "currency": "IRR",
  "breakdown": {
    "base_amount": 5900000,
    "distance_amount": 5400000,
    "stop_fee": 500000,
    "rate_per_km": 12000,
    "commission_rate_bps": 1000
  },
  "created_at": "2026-06-10T12:00:00+00:00"
}
```

GET `/api/v1/pricing/shipments/{shipment_id}/quotes/latest`

Latest quote, or **404** / `QUOTE_NOT_FOUND` .

POST `/api/v1/pricing/shipments/{shipment_id}/confirm-snapshot`

Requires header `Idempotency-Key` .

Takes the latest quote and freezes it. Rules:

- No quote → **404** / `QUOTE_NOT_FOUND`
- Snapshot already exists for this shipment → **409** / `SNAPSHOT_ALREADY_EXISTS` (even with a different idempotency key)
- Same idempotency key, same operation replayed → **200**, same snapshot, **no second event**
- Same key reused for a different shipment (or conflicting context) → **409** / `IDEMPOTENCY_CONFLICT`
- First successful confirm → **201**, write snapshot, publish event

Response:

```
{
  "snapshot_id": "...",
  "shipment_id": "...",
  "quote_id": "...",
  "quote_version": 1,
  "gross_amount": 5900000,
  "commission_amount": 590000,
  "driver_net_amount": 5310000,
  "currency": "IRR",
  "confirmed_at": "2026-06-10T12:05:00+00:00"
}
```

GET `/api/v1/pricing/shipments/{shipment_id}/snapshot`

Confirmed snapshot, or **404** / `SNAPSHOT_NOT_FOUND` .

GET `/health` and `/ready`

`/health` : process is up. `/ready` : Postgres reachable, else **503**.

Event on confirm

First successful confirm only (not idempotent replay): append to Redis stream `asanbar:events` .

One field `payload` , JSON inside:

```
{
  "event_id": "...",
  "event_type": "price_snapshot.created",
  "occurred_at": "2026-06-10T12:05:00+00:00",
  "correlation_id": null,
  "data": {
    "shipment_id": "...",
    "price_snapshot_id": "...",
    "quote_id": "...",
    "gross_amount": 5900000,
    "commission_amount": 590000,
    "driver_net_amount": 5310000,
    "currency": "IRR"
  }
}
```

For tests, `fakeredis` is fine if you document it.

Database

Schema `pricing` . You'll probably want tables along the lines of `pricing_quotes` , `price_snapshots` , something for idempotency keys, and `commission_rules` for the seed. Name things how you like. Just explain in the README.

Alembic migrations included. We should be able to run `alembic upgrade head` and go.

Worth putting a unique constraint on one snapshot per shipment.

Tests

`pytest` , one command to run everything. Document it in the README.

At minimum we'd like to see:

- Unit test on commission math (including the gross = commission + driver_net check)
- Unknown rate card → 422
- Confirm twice with the same `Idempotency-Key` → one snapshot, one event
- Second confirm on a shipment that already has a snapshot → 409

Postgres in Docker, testcontainers, whatever. Your call, just say how to run it.

What to send back

GitHub/GitLab link or zip. Include:

- Code (`app/` , `tests/` , `alembic/` , usual layout)
- README: setup, env vars, run app, run tests, and a short note on how you handled idempotency + what you'd harden for prod
- `.env.example`
- `pyproject.toml` (or requirements + how you pin deps)

`docker-compose` for Postgres/Redis is optional but appreciated.

Don't build: JWT, commission admin CRUD, event consumers, K8s/CI, payment integration. If you're done early and want to add admin override or a consumer stub, mention it in the README. Won't hurt.

Submission

Please submit GitHub/GitLab link to Shahram Alizadeh.

Using Copilot/Codex/Cursor is fine. We'll walk through your code in the next round and expect you to own it.

Thanks,
Asanbar eng team